

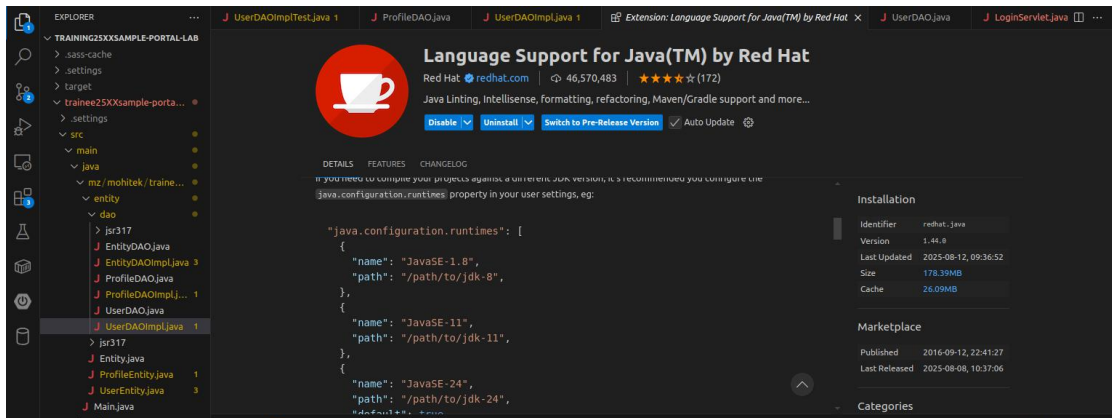
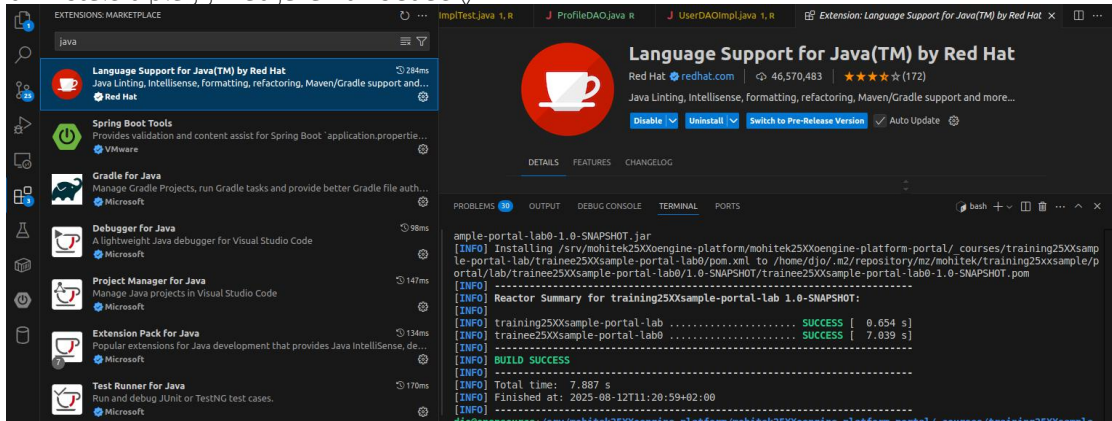
[Lab1]-Configuracao de Interoperabilidade entre sistemas e troca de mensagens por servicos RestFul.

data: 15/18.08.2025

status: *in progress*

Pre-requisito ambiente local:

- instale o VsCode (Visual Studio Code)
- instale o plugin do java no VsCode ()



- instale o sdkman (<https://sdkman.io>)
- instale o *java-8* e *maven-3.6.3* a partir do *sdkman*

Referencias:

- <https://jcp.org/en/jsr/detail?id=311>
- <http://download.oracle.com/otn-pub/jcp/jaxrs-1.0-fr-eval-oth-JSpec/jaxrs-1.0-final-spec.pdf>
- https://docs.openolat.org/archive_confluence/display/OO155EN/REST+API.html

Historico de alteracoes

Data	Item atualizado	
14.08.2025		2.3.1. Estrategia de Seguraca de acesso aos recursos de rede
		✓ Inclusao de novos mecanismos.
		✓ correcao do pacote <i>mz.a2e2</i> para <i>mz.mohitek</i>

2. training25XXsample-portal-lab1

Neste modulo e' onde se localizam todas as classes em *java* que garantem a integracao dos dados do nosso sistema com outras aplicacoes utilizando *webservices*.

2.1. Tecnologias por considerar

Os conceitos e tecnologias a serem consideradas neste modulo sao: Tecnologias de troca de mensagem sincronas.

Nas tecnologias de troca de mensagem sincronas podemos encontrar o Webservices baseado no protocolo SOAP (**JAX-WS** - Java Xml Webservice) e pelo **JAX-RS** (Java WebService RestFul) atraves da API JSR311/JSR340 - Java WebFilter.

Nota:

Neste curso de Desenvolvimento de aplicacoes utilizaremos o *RESTful*.

2.1.1.1. Funcionamento do JSR311-RESTful

Restful, e' uma tecnologia que permite realizar a troca de mensagens na camada de apresentacao atraves do protocolo Http entre outros. As mensagens mais comuns podem ser caracterizadas por dois tipos de media padrao:

- JSON
- XML

Nota:>

RESTful utiliza:

- i. URIs para identificar recursos
- ii. HTTP methods para manipular recursos (
 - a) GET - como operador de busca de conteudos
 - b) PUT - para criar novas instancias de objectos em bases de dados persistentes.
 - c) POST - para modificar estados de objectos existentes.
 - d) DELETE - para remover conteudos existentes em bases persisitentes.
 - e) MediaTypes - como formatos de ficheiros de troca de mensagens (ver 2.1.1)

2.2. Arquitectura

A arquitectura utilizada e' em camadas.

2.3. Seguranca de acesso utilizando chamadas HTTP

Neste modulo realizaremos testes de conexao e de acesso aos dados atraves de chamadas *Webservices* por *Http*. Estas chamadas por razoes de seguranca devem ser filtradas utilizando diferentes estrategias (Ex. combinancoes entre as entidades ProfileEntity e RoleEntity)e a utilizacao. Em *LpJava* a especificacao que cuida deste processo e a JSR340 - Servlet Filter.

2.3.1. Estrategia de Seguraca de acesso aos recursos de rede

O mecanismo de seguranca a ser implemetado neste module obedecera dois niveis:

1. O primeiro nivel acontece quando o *servlet filter* que colecta todas as chamadas REST API determinam qual **URI**(*Universal Resource Identification*) esta disponivel para todos os usuarios. (**/api, /auth**)
2. O segundo nivel acontece quando em todos endpoints no REST, verifica-se se o utilizador esta ou deve ser autenciado para ter acesso ao recurso disponivel.

Nota:>

Para manter a autenticacao do utilizador validada por um periodo de tempo apartir de uma quantidade de pedidos (*request*), o filtro pode utilizar duas tecnicas:

1. (*Session*¹*Cookies*²)³em todos os pedidos. A vantagem desta tecnica e' que pode nao ser necessario a criacao de *sessions* para cada pedido de conexao.
2. o filtro adiciona sempre um cabecalho no HTTP Header

(X-TRAINEE25XX**DEMO**YYY-TOKEN) para a conexao de resposta (*response*)

¹ Informacao utilizada para identificar o representante de uma conexao solicitada. Esta pode ser partilhada por outras solicitacoes criadas pelos mesmo utilizador atraves do mesmo cliente.

² Conjunto de informacoes criadas automaticamente e controladas pelo seu agente de busca (ex. Firefox). Estas informacoes ficam armazenadas por meses ou anos pois sao permanentes.

³ Utiliza os conceitos anterior todavia ele usa o mecanismo *stateless*, onde as informacoes sao armazenas que uma requisicao for solicitada e logo que esta expira todos os dados sao removidos do agente (brower) automaticamente.

Actividades

S2A0. Importe o projecto `training25XXsample-portal-lab` para o *Netbeans* e acesse ao modulo `training25XXsample-portal-lab1`.

- a. altere o `pom.xml`

S2A1. Crie as seguintes entidades ([XEndpoint](#)) no pacote `mz.mohitek..trainee25xxshifu.appdevelopment.lab1.jsr311.endpoint.*`

- a. altere o `pom.xml`

S2A2. Crie as seguintes entidades ([XFilter](#)) no pacote `mz.mohitek.trainee25xxshifu.appdevelopment.lab1.jsr340.filter.*`

S2A3. Crie as seguintes configuracoes de seguranca: